

CS 407: Artificial Intelligence

FIRST SEMESTER 2024-25



ASSIGNMENT II

DATE: 16/11/24

TOPIC

Shallow Q-Learning Network for Tic Tac Toe Game

SUBMITTED BY

Yatee Singh 2021A8PS2534G

Introduction

Problem Overview:

The objective of this assignment is to understand the Deep Q-Learning algorithm in Reinforcement Learning. Instead of utilizing a deep neural network with multiple layers, we will implement a simplified version using a shallow neural network with just three layers.

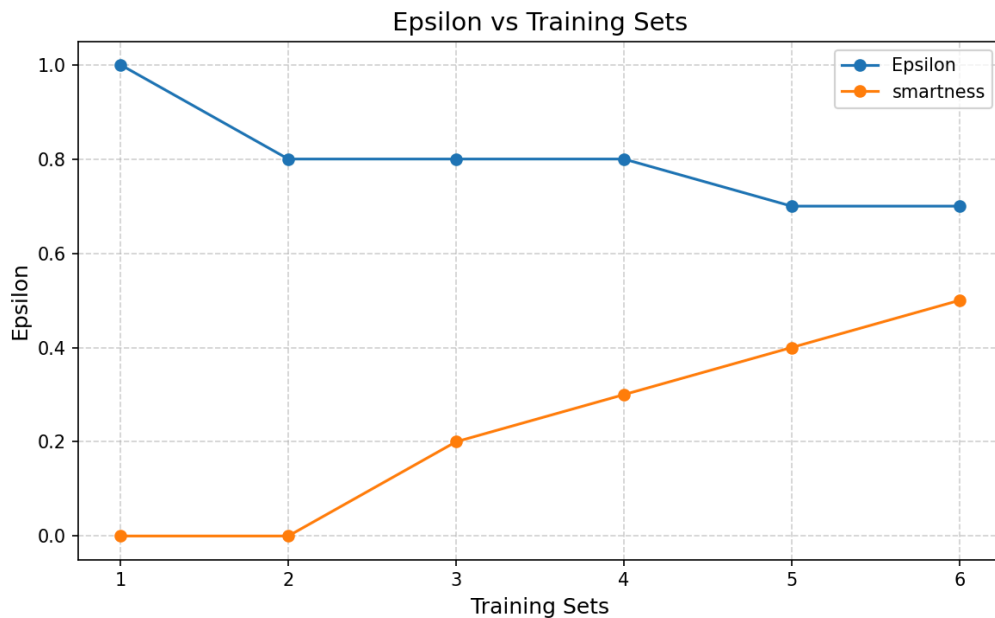
The task is to train "PlayerSQN" to play TicTacToe using this algorithm. PlayerSQN will learn to compete against a simulated opponent (Player1) whose level of smartness can be adjusted through a configurable probability parameter. Training involves collecting gameplay data in a replay buffer, which captures PlayerSQN's experiences. These experiences are then used to iteratively update the Q-values $Q(s,a)$ enabling PlayerSQN to make increasingly effective decisions over time.

Methodology

1)Populating the replay buffer:

- Generated a list of (s,a) tuples during one episode, then derived the (s,a,r,s') tuples from list. The r value was 1 for the last tuple if playerSQN won else -1, to avoid draws. [s:state matrix containing players' moves, a: action, r:reward, s': new state after action on s]
- Stored around 1000 episodes that was approximately 3500 elements in the replay buffer for one set of training (5 Epochs) the neural network.
- During this collection of data also playerSQN chose action according to the epsilon-greedy algorithm.

- This epsilon was increased gradually, starting from 1 in the beginning. Values of Epsilon and Smartness probability used over training sets:



2) Initializing the Neural Network:

Network Summary-

Model: "sequential"

Layer (type)	Output Shape	Param #
dense_1 (Dense)	(None, 64)	4,160
dense_2 (Dense)	(None, 9)	585

Total params: 5,387 (21.05 KB)
Trainable params: 5,385 (21.04 KB)
Non-trainable params: 0 (0.00 B)

Loss function: Mean squared error, Optimizer: Adam

Training Parameters-

```
BATCH_SIZE =65  
GAMMA = 0.99  
EPOCHS = 5  
LEARNING_RATE=0.001
```

3)Training Process:

For each Epoch, I went through the replay buffer in the batch sizes of 65. For tuple in each mini-batch, Q values of state s and Q values for next state s' was calculated, then this was used to calculate the Q_target value using the formula,

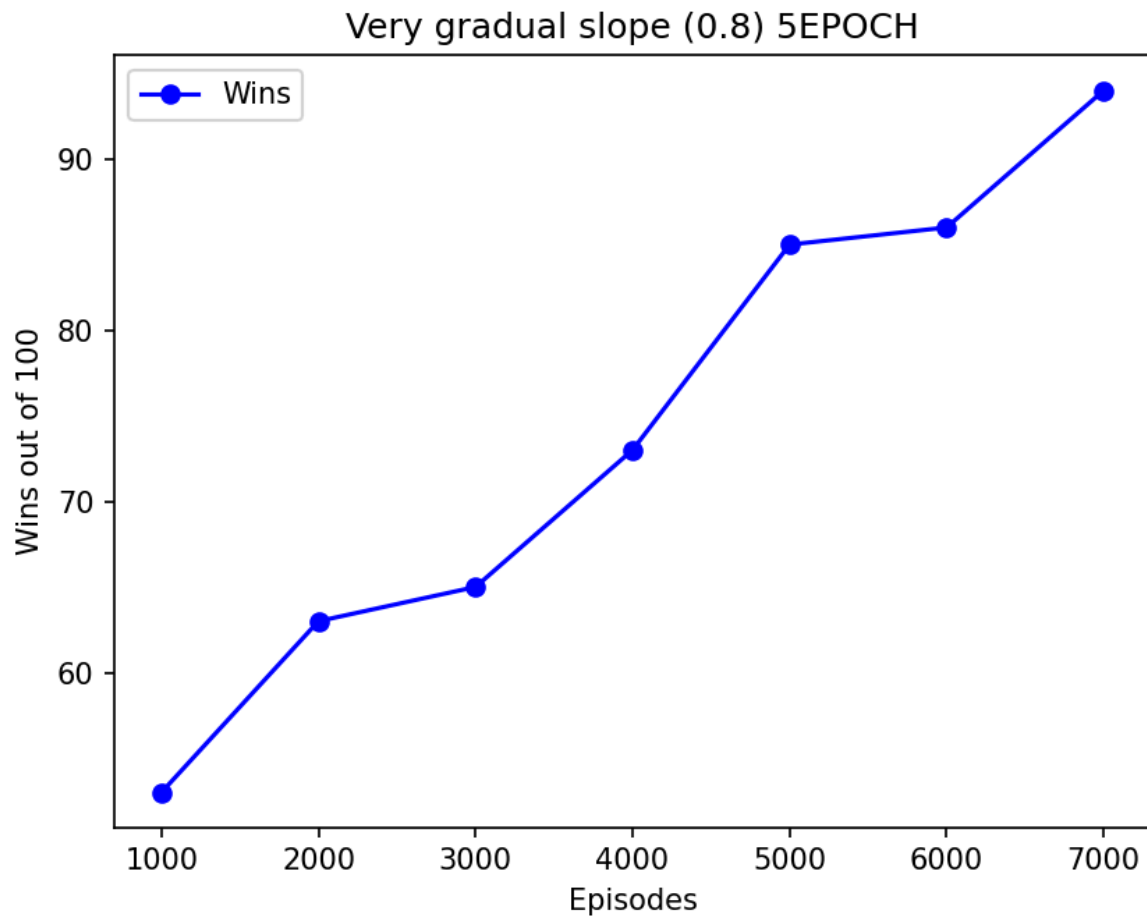
$$Q_{\text{target}}(s, a) = r + \gamma \max_{a'} Q(s', a')$$

Then this Q_target was used to train the neural network,by minimizing the difference between Q_target and Q.

Training was done for over 7000 episodes, increasing the smartness probability of Player1 gradually.

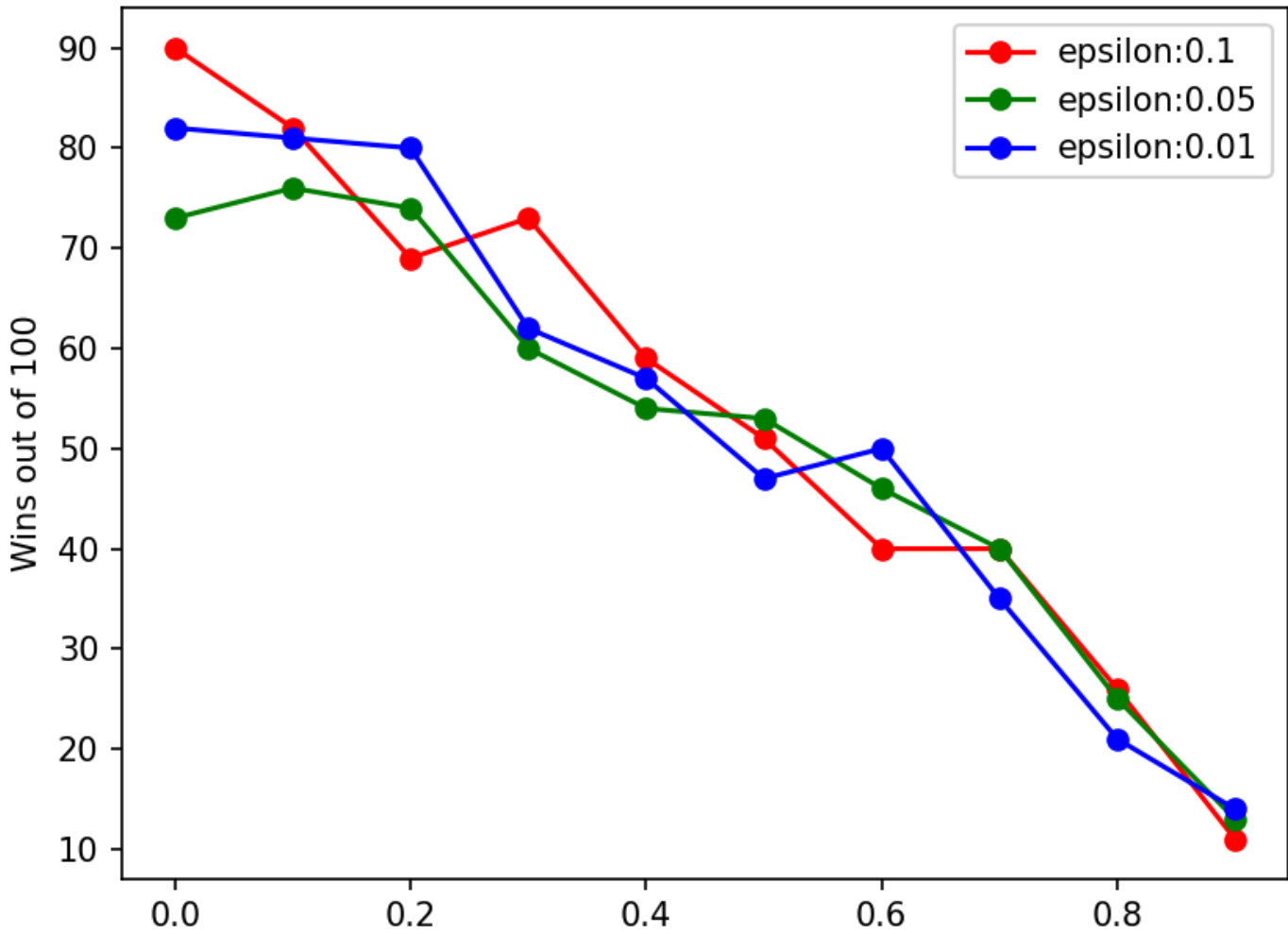
Results

- 1) Improvement in the performance of the model over episodes (smartness =0)



2) Performance of the final model over increasing smartness

Result Graph: Performance over different smartness probability



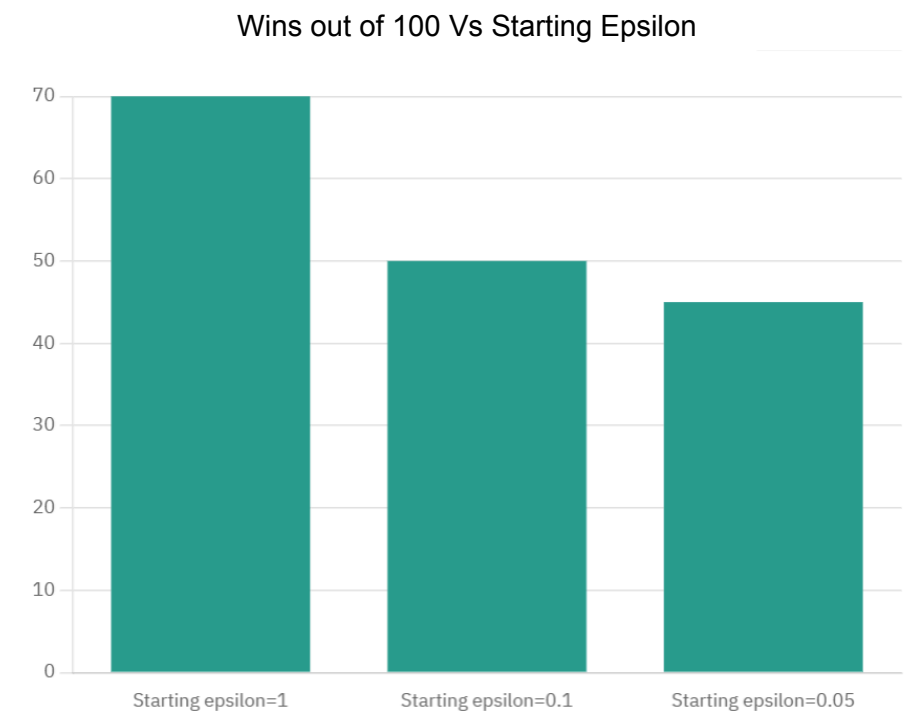
Discussion

(All graphs are for smartness=0 by default)

Let's discuss the factors that were tweaked in an attempt to improve the model's performance.

1) Initial training with epsilon=1

Initially, I did not pay much attention to the initial value of epsilon used by PlayerSQN while collecting data for the first iteration of training, but soon realized that models trained with epsilon=1 consistently outperformed the others. Below graph shows the performance of models after 1000 episodes of training with different epsilon.



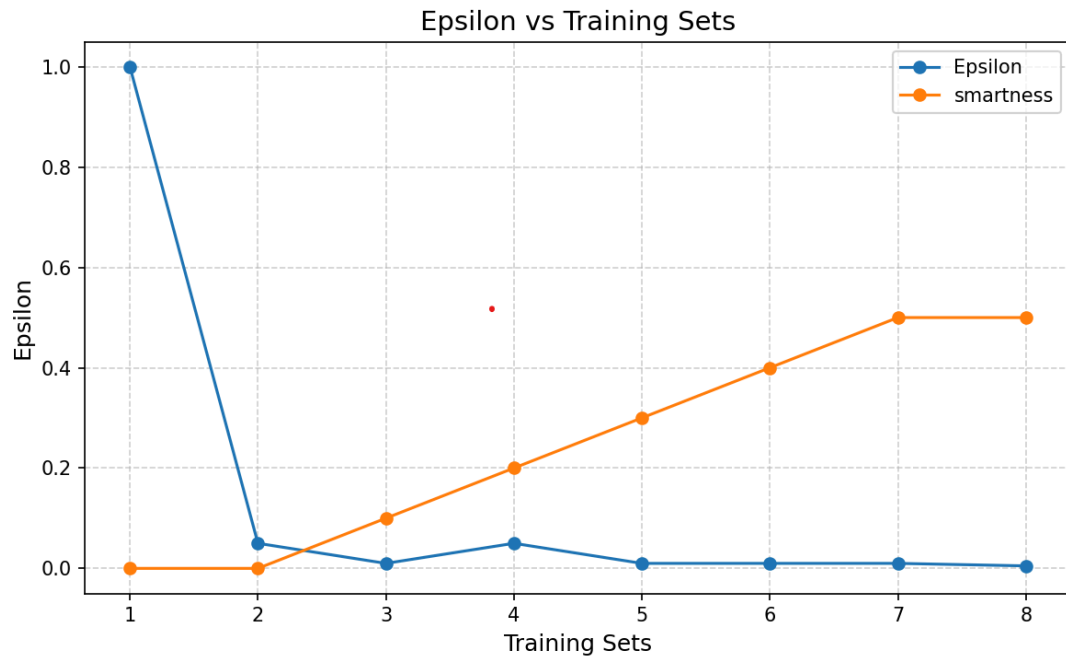
2) State Representation:

Models trained using 1,0,-1 as PlayerSQN,Empty and Player1 respectively in the state representation,performed slightly better than 0,1,2.

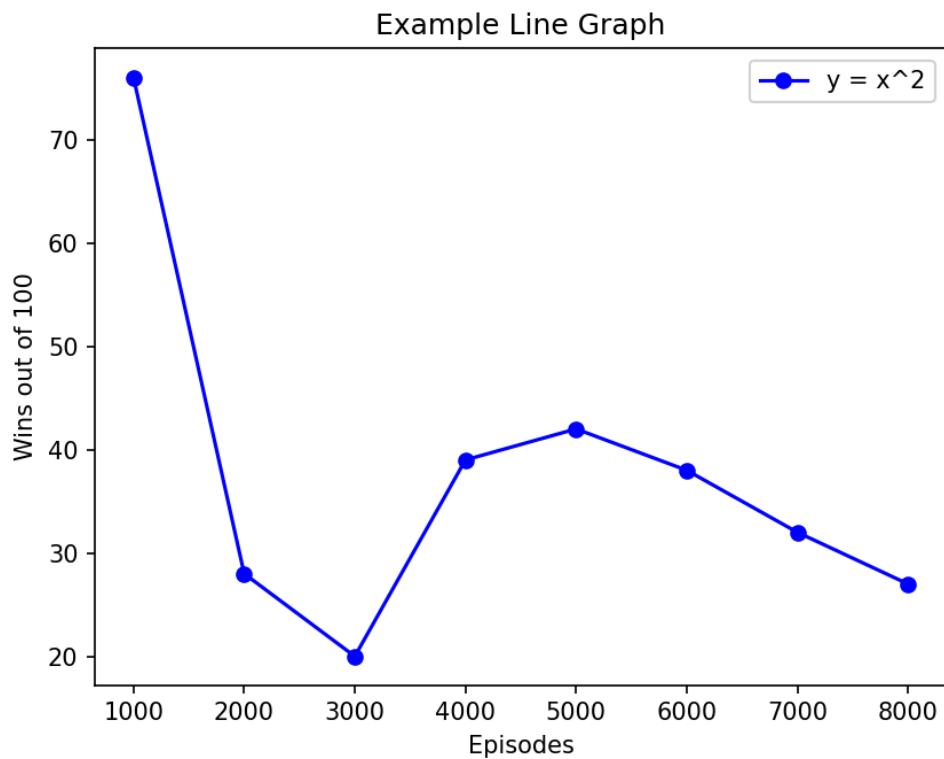
3)Rate of Epsilon Decay during Training

Not only the initial value of epsilon,but also the rate at which it was decreased over the training highly affected the performance.

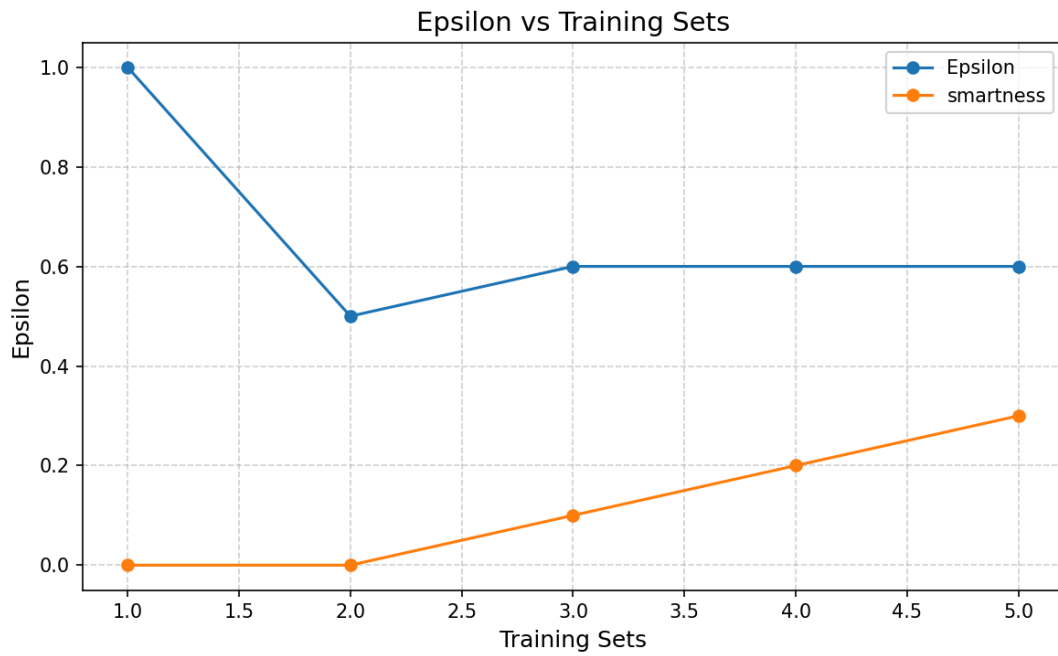
a) High Decay of Epsilon over Training sets: The model's **performance deteriorated** in this case



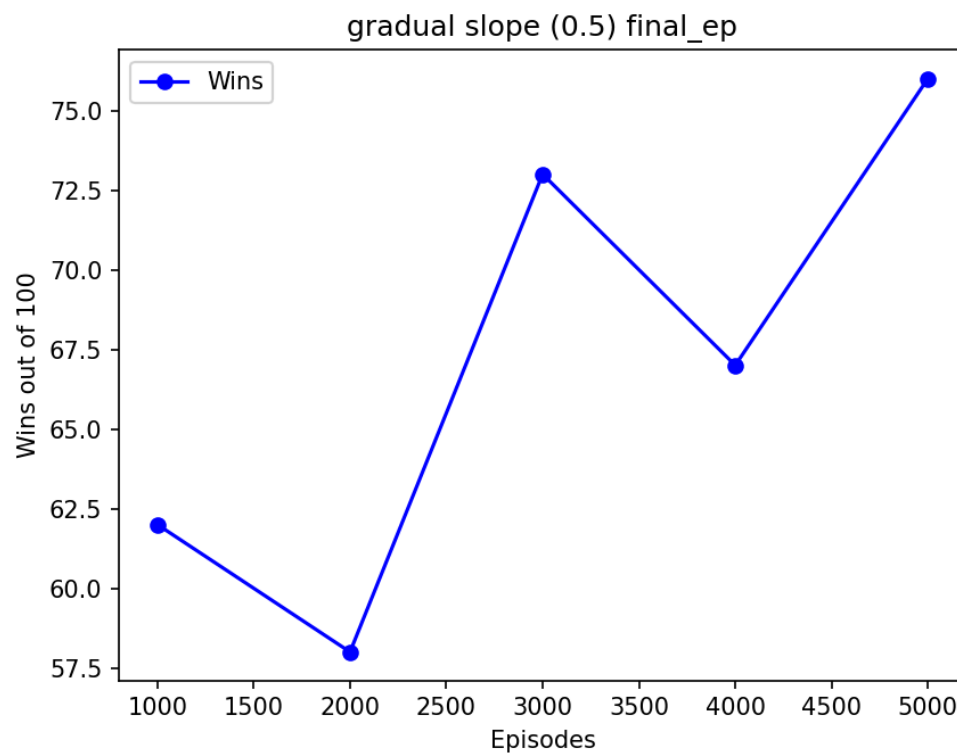
Model performance over training sets of 1000 episodes



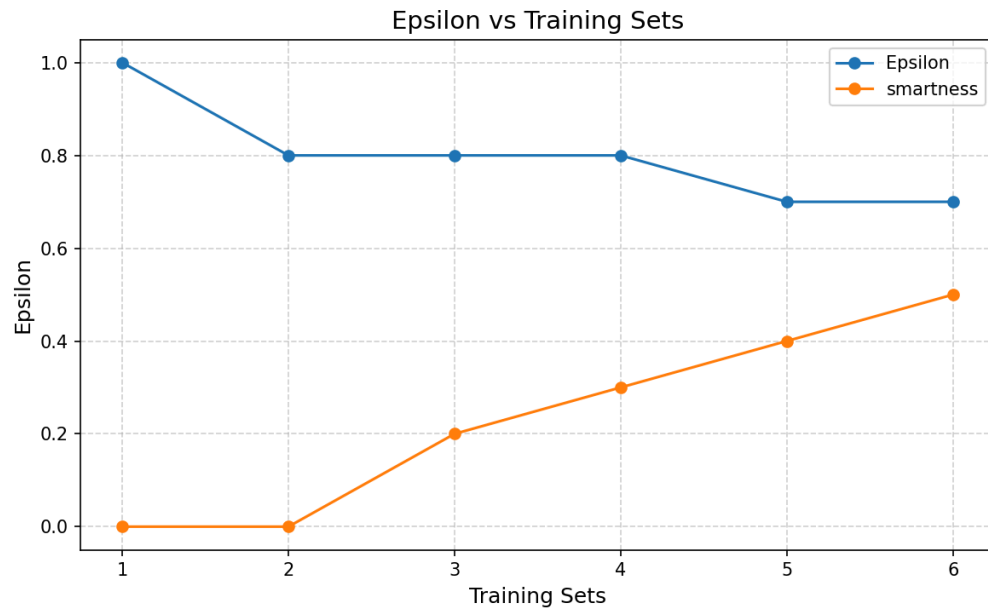
b) Lower Decay Rate of Epsilon over Training sets: The model **started improving** over time



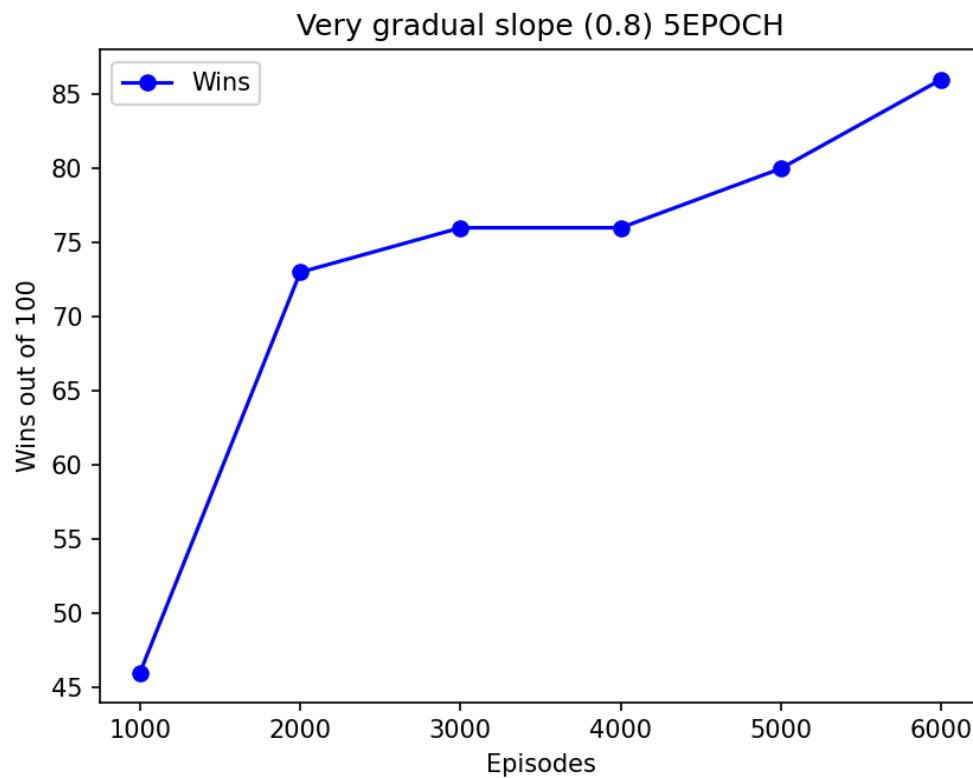
Model performance over training sets of 1000 episodes



c) Very Low Decay Rate of Epsilon over Training sets: The model showed **significant improvement**



Model performance over training sets of 1000 episodes

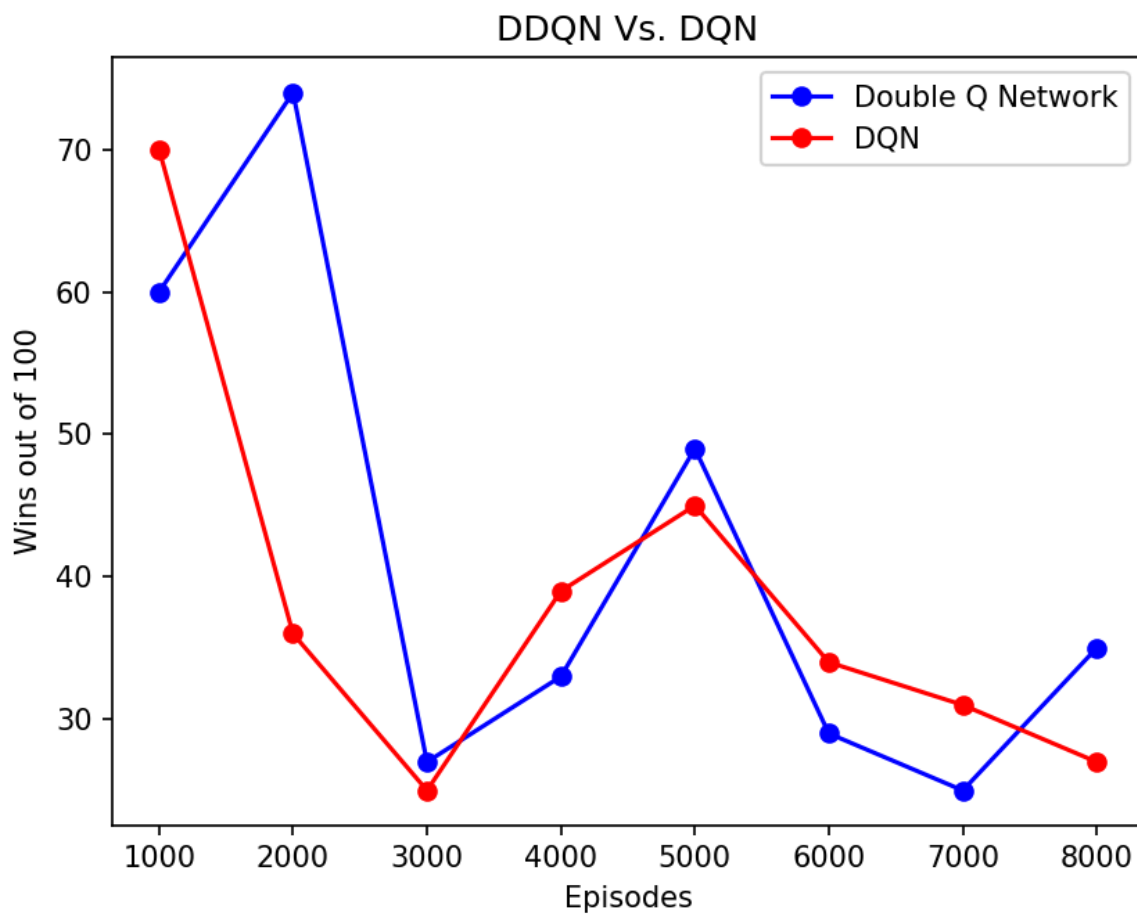


4) Using two SQNs in the Algorithm

In this algorithm, the action with the maximum Q value is selected using one network (the “selection network”), and the Q value for this action is evaluated using a separate network (the “target network”) and after time steps T, they are synced by copying the weights of selection network to target network.

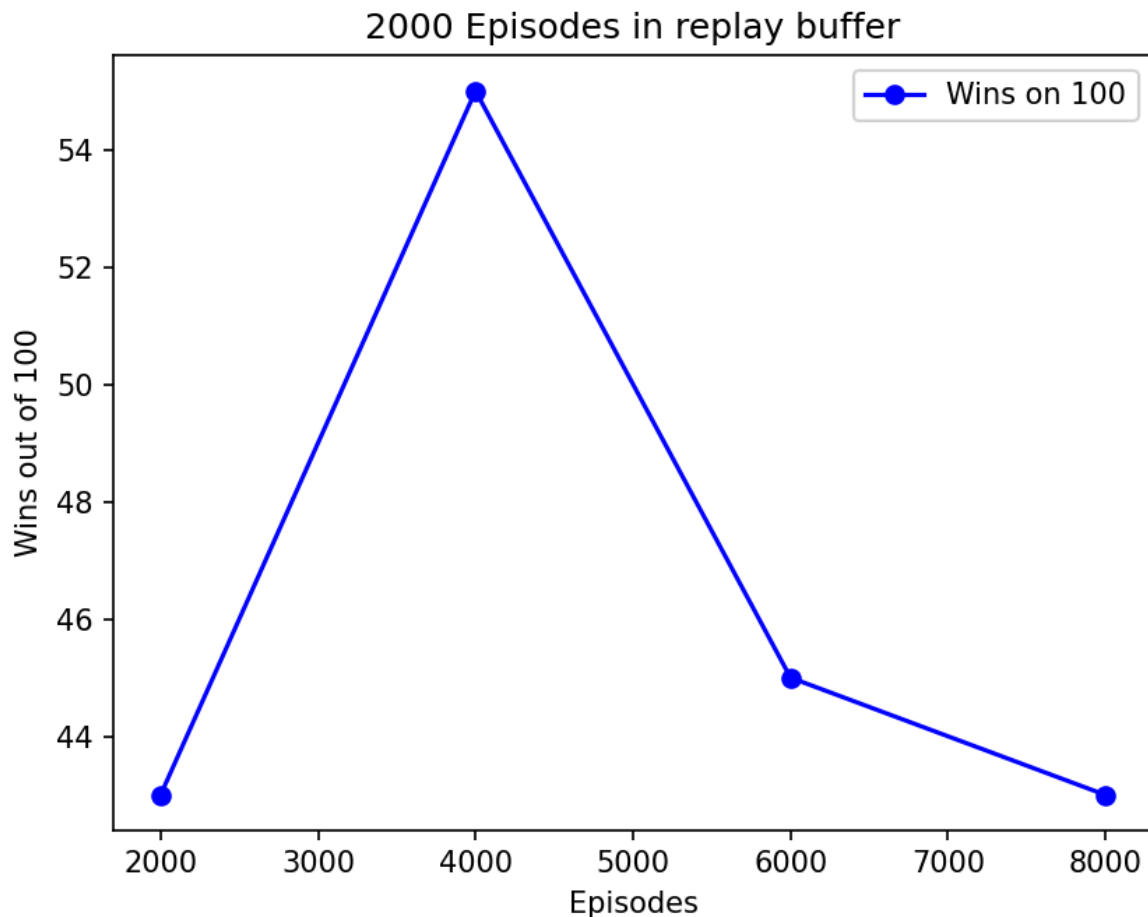
This algorithm was not implemented in the final model because there was **no significant improvement** over the normal algorithm.

The Graph shows the performance of the two algorithms over training episodes.



5) Number of episodes in replay buffer

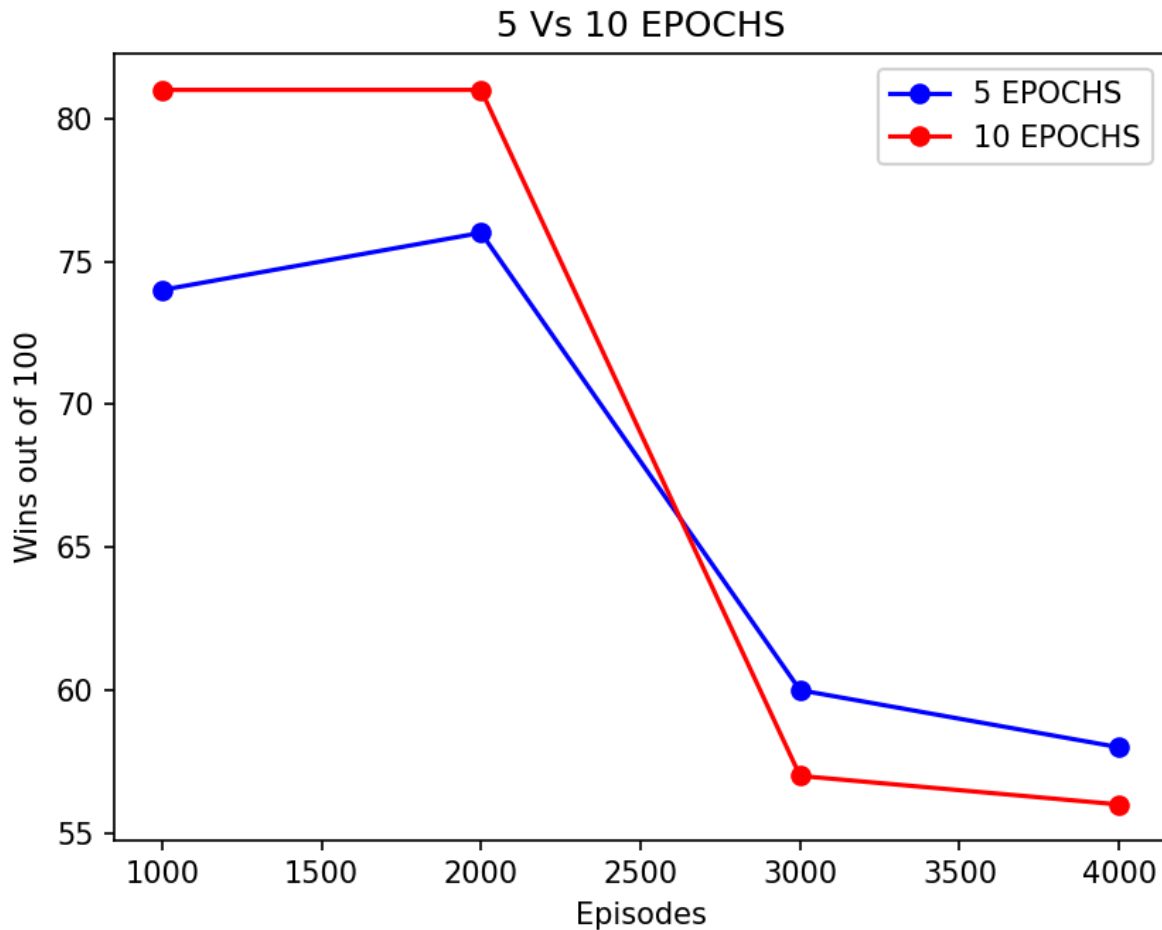
There was **not much improvement** in a model, when it was trained with double the number of episodes in the replay buffer as others. The below graph was taken for high decay rate in epsilon, it is very similar the 1000 episodes graph in section 3(a)



Also, training the model with more episodes keeping epsilon constant also did not improve the model, in some cases it again deteriorated.

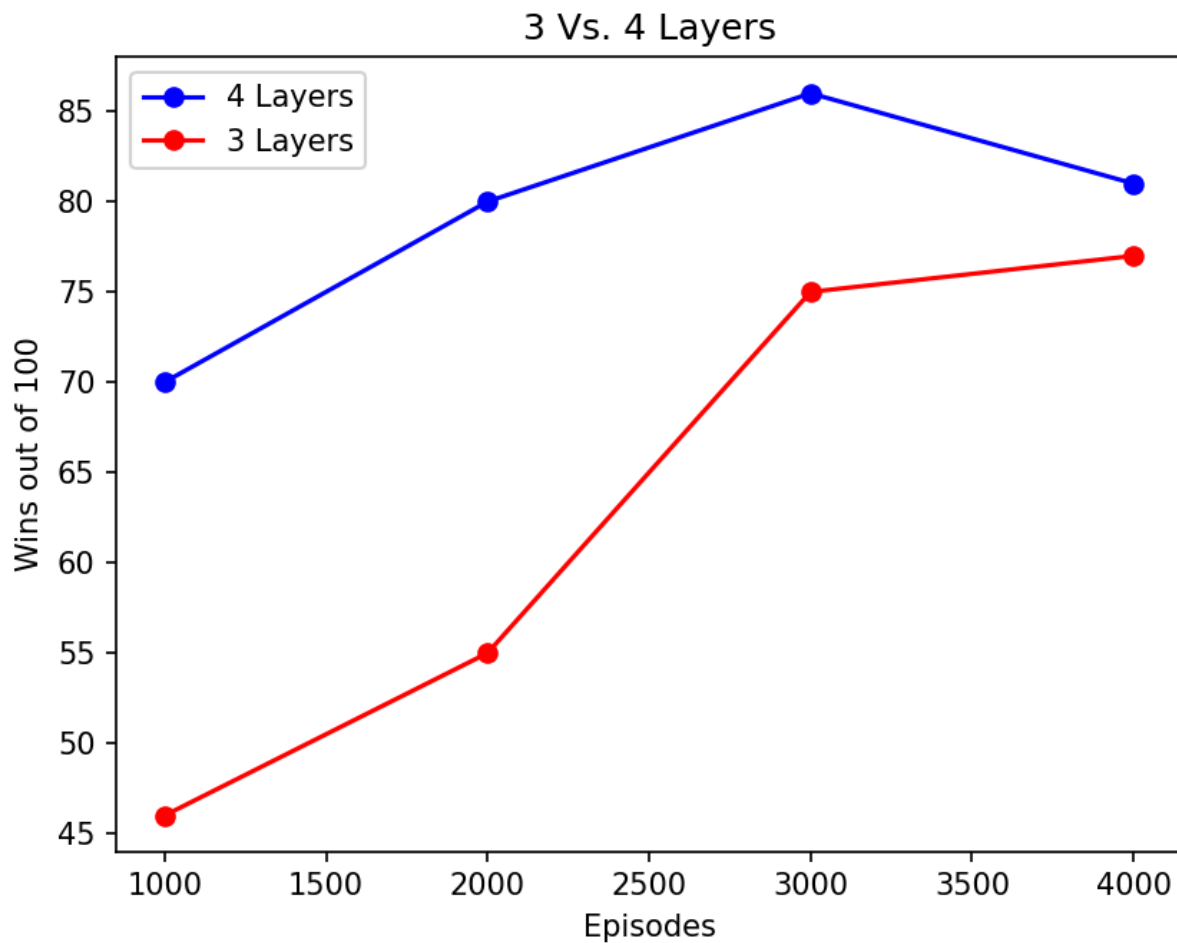
6) Number of Epochs

The performance of model was similar for 10 and 5 Epochs, so 5 Epochs were used in the final implementation



7) Adding a Layer in the Neural Network:

The 4 layer model **did well initially**, but after 4000 episodes of training, its performance started converging with the 3 Layer. After the dip in performance, the model was not tested further.



Conclusion

The Deep Q Learning Algorithm of Reinforcement Learning was used to train a model to play Tic Tac Toe against the simulated player. The model shows a success rate of 85% while playing in the random mode. The most important factor that was missing in the initial stages of making the model was the right decay factor of epsilon and its initial value. Increasing the number of episodes was effective only when smartness or epsilon also increased for the training data.

Possible Improvements:

1. **Tweaking the neural network parameters:** Learning rate, optimizer and loss functions were not tweaked to improve the performance.
2. **Deeper Neural Network:** More layers were not added due to the limited knowledge of neural networks.
3. **More training with greater smartness probability:** The model is still not very accurate while playing with smartness probability greater than 0.4
4. **Implementing DDQN algorithm :** When this algorithm was implemented while testing, the target and policy network sync time was not varied enough to retrieve the optimal performance of the DDQN algorithm, which resulted in it performing the same as normal algorithm.